

머신러닝 10 교시

군집분석

K-Means · 계층적 · GMM · DBSCAN

학습 목표

- 군집분석이 분류와 어떻게 다른지 설명할 수 있다 (라벨의 유무, 학습 형태, 평가 방식).
- "동질성" 과 "이질성" 이 군집의 좋고 나쁨을 어떻게 정의하는지 설명할 수 있다.
- K-Means 알고리즘의 5 단계와 손실함수(WCSS) 의 의미를 이해한다.
- 초기값 의존성과 그 회피 전략(`n_init`, `k-means++`) 을 설명할 수 있다.
- 엘보우 기법과 실루엣 계수로 적절한 군집 수 k 를 결정할 수 있다.
- 계층적 군집분석의 응집형 절차와 네 가지 linkage(single/complete/average/ward) 를 비교할 수 있다.
- 덴드로그램을 읽고 적절한 cut height 를 정해 군집을 추출할 수 있다.
- Mean Shift · GMM · DBSCAN · BIRCH · Affinity Propagation · OPTICS 등 주요 군집화 알고리즘을 가정과 적합 데이터 형태로 비교할 수 있다.
- scikit-learn 으로 각 알고리즘을 구현하고 결과를 시각화·해석할 수 있다.

선수 학습 — 유클리드 거리, 분산·평균의 의미, 데이터 표준화의 기초.

사용 도구 — Python(scikit-learn · scipy · matplotlib).

1. 군집분석이란 무엇인가

지도학습은 정답이라는 등불을 들고 길을 걷는다. 그런데 현실의 데이터는 등불 없이 쏟아진다. 고객 수십만 명을 어떻게 묶을지, 무수히 많은 문서를 어떤 주제로 분류할지, 아무도 답을 알려주지 않는다. 이럴 때 데이터 스스로 비슷한 것끼리 무리를 짓도록 시키는 비지도학습 기법이 바로 군집분석(clustering) 이다.

데이터를 마주했을 때 가장 흔한 질문 중 하나가 "이 안에 어떤 그룹이 숨어 있는가?" 이다. 정답 라벨이 없는 상황에서 서로 비슷한 관측치를 묶고, 서로 다른 묶음 사이에 의미 있는 차이가 있는지 보는 작업이 군집분석이다. 비즈니스 현장에서 고객 세분화·이상 탐지·문서 그룹화 등에서 가장 먼저 손이 가는 도구이며, 머신러닝 비지도학습의 대표 주제이기도 하다.

1.1 정의 — 동질성과 이질성

군집분석(clustering)이란 전체 데이터를 일관성과 응집성이 높은 세부 그룹으로 나누는 작업이다. 한 군집 안의 개체들은 서로 비슷하고(동질성), 서로 다른 군집에 속한 개체들은 서로 다르다

(이질성). 이 두 가지를 어떻게 "수치로" 측정할 것인가가 군집분석의 모든 알고리즘이 답해야 하는 핵심 질문이다.

동질성(homogeneity) – 같은 군집 안의 점들이 서로 얼마나 가까운가.

보통 "군집 내 분산" 또는 "중심까지의 평균 거리" 로 잰다.

이질성(heterogeneity) – 다른 군집의 점들이 서로 얼마나 멀리 떨어져 있는가.

"군집 간 거리", "중심 간 거리" 로 잰다.

좋은 군집 = 동질성 ↑, 이질성 ↓.

1.2 분류(Classification) vs 군집화(Clustering)

두 용어는 비슷해 보이지만 출발점이 완전히 다르다. 분류는 정답 라벨이 미리 주어진 지도학습이고, 군집화는 라벨이 없는 비지도학습이다. 분류는 "고양이/개" 같은 범주를 새 사진에 붙이는 작업이며, 군집화는 라벨이 없는 사진 1만 장을 보고 "비슷한 것끼리 묶어 보라" 는 작업이다.

- 분류: 범주의 수와 각 개체의 범주가 사전에 알려져 있다. 입력 변수로부터 라벨을 유추한다 (지도학습).
- 군집화: 군집의 수·속성이 사전에 알려져 있지 않다. 데이터 안의 구조를 스스로 찾아낸다 (비지도학습).
- 공통점: 둘 다 "새 개체를 어느 그룹에 넣을지" 라는 결정을 내린다.
- 차이점: 분류는 정답이 있으므로 정확도(accuracy)로 평가하고, 군집화는 정답이 없으므로 실루엣 같은 내부 지표로 평가한다.

1.3 군집분석의 활용 예

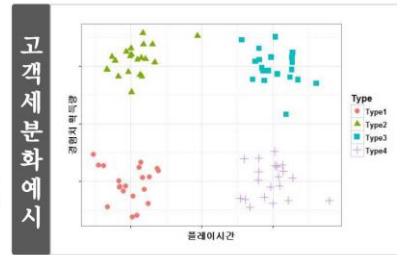
군집분석은 "누구를 어떻게 묶을 것인가" 라는 질문에 답하는 도구로 자주 쓰인다. 대표 사례 두 가지를 본다.

- 고객 세분화: 우수 고객의 인구통계·생활 패턴을 파악해 맞춤 관리, 신상품 판촉, 교차 판매의 목표집단을 정의한다.
- 이상 탐지: 정상 행동의 군집에서 벗어난 개체를 비정상으로 표시한다. 카드 부정 사용 탐지·서버 침입 탐지 등에 활용된다.

주요 예시

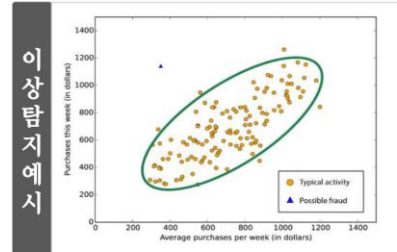
고객 세분화

우수 고객의 인구통계적 요인과 생활패턴 등을 파악함으로써 개별 고객에 대한 맞춤 관리
신상품의 판촉이나 교차판매를 위한 목표 집단을 설정



이상 탐지

정상적인 거래에 대한 데이터에 대하여 군집 분석을 실시하여 군집을 찾아서 신규 거래가 군집과의 거리가 크면 자동적으로 불량품으로 인식



Source : <https://m.blog.naver.com/PostView.nhn?blogId=gkenq&logNo=10188552802&proxyReferer=https%3A%2F%2Fwww.google.co.kr%2F>

[그림 1] 고객 세분화와 이상 탐지 — K-means 의 대표 활용 예

한 실제 사례로, 화장품 앱 "언니의 파우치" 는 사용자 군집화를 통해 주요 고객층이 사실은 10대 후반에서 20대 초반이라는 것을 발견했다. 분석 전에는 막연히 20대 후반이 주력이라고 추정했지만, 군집분석 결과를 보고 신제품 개발과 이벤트 전략 자체가 바뀌었다. 이처럼 군집분석은 데이터에서 "몰랐던 그룹" 을 끄집어내는 데 강하다.



[고객세분화]

그 결과, 화장품에 관심을 갖기 시작한 중고등학생은 다른 그룹(Group1)보다는 앱 내 활동이 구매에 미치는 영향이 컸습니다. 이들의 구매를 유도하기 위해서는 팔로워 및 팔로잉 기능을 강화하고 인적 네트워크 활성화하는 등 앱 내 활동을 활성화시키는 방안이 필요했습니다.

또 이벤트에만 관심있는 30대 이상 진짜 언니들(Group5)은 이벤트 참여를 주목적으로 언니의 파우치 앱을 사용하고 있었습니다. 이들은 10만원 이상씩 구매하는 톡 큰 고객이었죠. 이들의 구매를 더욱 늘리기 위해서는 이벤트를 차별적으로 적용할 방안이 필요했습니다.

Source : http://www.edaily.co.kr/news/news_detail.asp?newsId=01521926615928592&mediaCodeNo=257

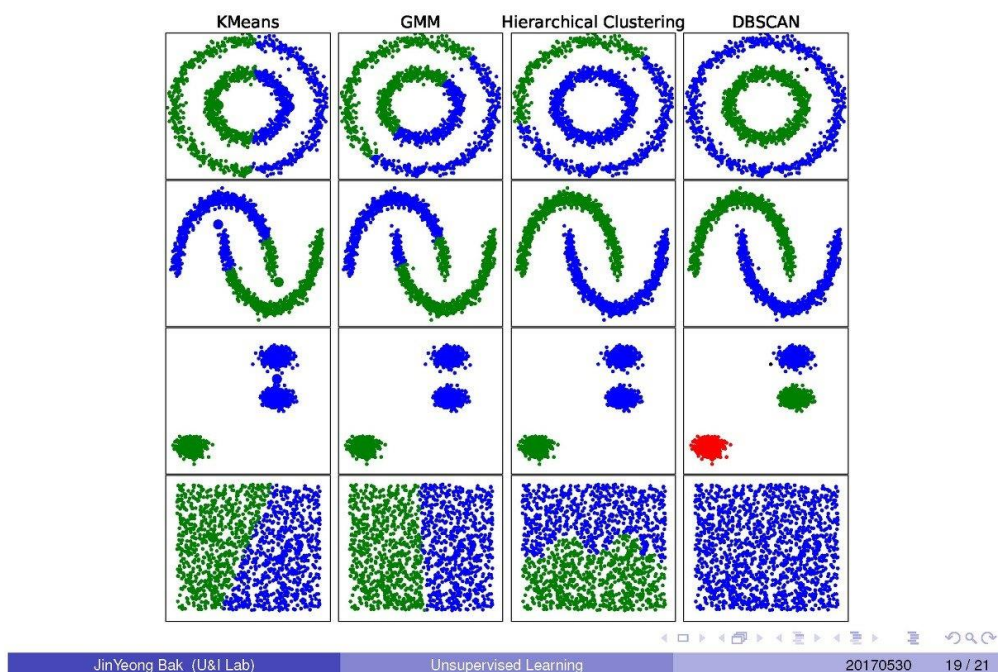
[그림 2] 화장품 앱 사용자 군집화 사례 — 데이터가 드러낸 진짜 주력 고객층

1.4 군집분석 알고리즘의 네 가지 부류

같은 "비슷함" 이라도 시각이 다르면 답이 달라진다. 아래 네 부류는 각자 다른 가정 위에 서 있다.

1. 거리 기반 (Centroid-based) – K-Means 처럼 중심점에서 가까운 점을 한 군집으로.
가정: 군집은 구형이고 크기가 비슷하다. 빠르고 단순하지만 K 를 직접 줘야 한다.
2. 밀도 기반 (Density-based) – DBSCAN · OPTICS 처럼 점이 뭉쳐있는 영역을 군집으로.
가정: 군집의 모양에 제약이 없다. 이상치를 자연스럽게 가려낸다.
3. 분포 기반 (Distribution-based) – GMM 처럼 군집을 확률 분포로 모델링.
가정: 각 군집은 가우시안 같은 특정 분포를 따른다. 타원형 경계와 소속 확률을 준다.
4. 계층 기반 (Hierarchical) – Agglomerative · BIRCH 처럼 점 → 작은 군집 → 큰 군집.
가정: 데이터에 트리 구조가 있다. 덴드로그램으로 시각화하기 좋다.

Summary



[그림 3] 여러 군집화 알고리즘의 비교 — 데이터 모양에 따라 잘 맞는 알고리즘이 다르다.

1.5 어떤 알고리즘을 골라야 하는가

"가장 좋은 알고리즘" 같은 것은 없다. 데이터의 모양과 목적이 결정한다. 일반적인 가이드라인은 다음과 같다.

- 계층 구조가 있는 데이터(생물 분류, 문서 트리 등) — Agglomerative · BIRCH.
- 중심이 명확하고 군집 크기가 비슷한 데이터 — K-Means.
- 한 점이 여러 군집에 걸쳐 있을 수 있는 데이터 — Gaussian Mixture Model.
- 이상한 모양이고 이상치가 있는 데이터 — DBSCAN · OPTICS.
- 초기 군집 수를 알기 어려운 경우 — Mean Shift · DBSCAN · Affinity Propagation.

팁 박스 – 군집분석은 "정답이 없는 문제" 다.

여러 알고리즘을 돌려 보고, 도메인 지식으로 결과를 해석하는 것이 옳은 길이다.

실루엣 계수처럼 군집의 품질을 양적으로 재는 도구도 함께 활용하자.

2. K-Means 군집분석의 원리

군집분석에서 가장 먼저, 그리고 가장 많이 등장하는 알고리즘이 K-Means 다. 1957 년 벨 연구소에서 처음 제안된 이래 60 여 년이 지났지만 여전히 첫 시도로는 K-Means 가 기본이다. 이유는 단순하다 — 빠르고, 이해하기 쉽고, 대부분의 경우 그럭저럭 잘 동작한다. 아이디어는 단순하다. 각 군집을 대표하는 점인 중심값(centroid) 을 두고, 모든 점을 가장 가까운 중심에 할당하고, 할당된 점들의 평균으로 중심을 갱신하는 작업을 반복한다.

2.1 알고리즘 개요 – 다섯 단계

K-Means 는 다음 다섯 단계로 정리된다. (1) 군집 수 k 를 결정하고 초기 중심값 k 개를 잡는다. (2) 각 점을 가장 가까운 중심에 할당한다 — 군집 할당 단계. (3) 각 군집의 평균을 새 중심값으로 한다 — 중심 갱신 단계. (4) 할당이 바뀌었는지 본다. 바뀌었다면 (2) 로 돌아가고, 바뀌지 않았다면 종료. (5) 보통은 "중심값이 거의 움직이지 않을 때" 라는 조건으로 종료를 판단한다.

k-means 군집분석의 수행절차



[그림 4] K-means 알고리즘의 5 단계 — 초기 중심 설정 → 할당 → 갱신 → 반복 → 수렴

K-Means 의 손실함수 – 모든 점에 대해 "자기 군집의 중심까지의 거리 제곱" 의 합을 최소화한다.

이를 WCSS (Within-Cluster Sum of Squares) 또는 inertia 라 부른다.

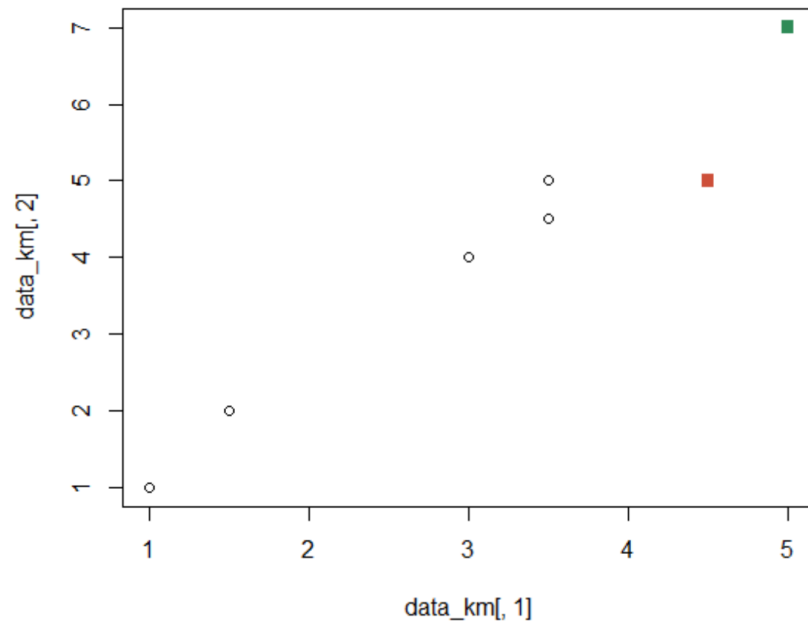
한 번의 반복마다 WCSS 는 단조 감소하므로 알고리즘은 반드시 수렴한다.

다만 "전역 최소" 가 아닌 "지역 최소" 에 빠질 수 있다.

2.2 한 걸음씩 따라가는 예제 ($k=2$)

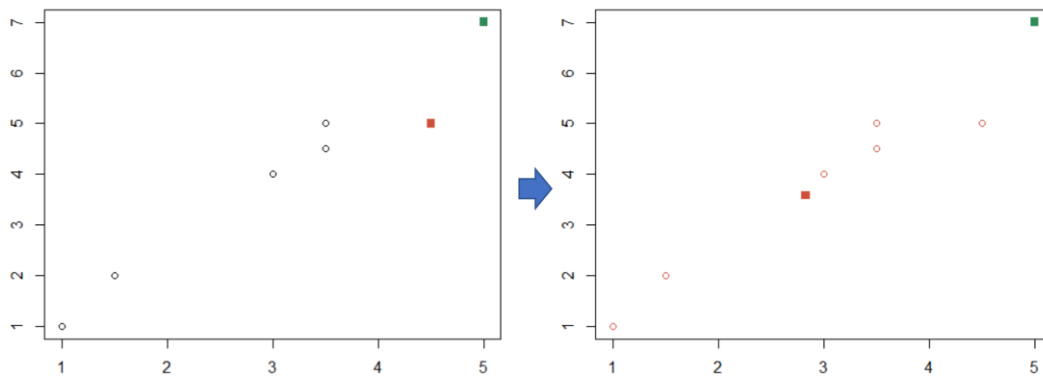
예시 데이터로 K-Means 가 어떻게 수렴하는지 단계별로 보자. 점 몇 개가 흩어져 있고 $k=2$ 로 설정했을 때, 임의의 두 점을 초기 중심으로 잡은 뒤 알고리즘이 어떻게 진행되는지 따라간다.

- 2. k개의 중심값과 각 개별 데이터간의 거리를 측정한다.
가장 가까운 군집에 해당 데이터를 할당한다.(군집 할당)



[그림 5] [1 단계] 랜덤하게 두 개의 초기 중심값 선택 (k=2)

- 4. 1번의 중심값과 3번에서 선택된 중심값이 변화가 없다면 멈춘다.
만약 데이터의 변화가 있다면 다시 1번부터 반복한다.

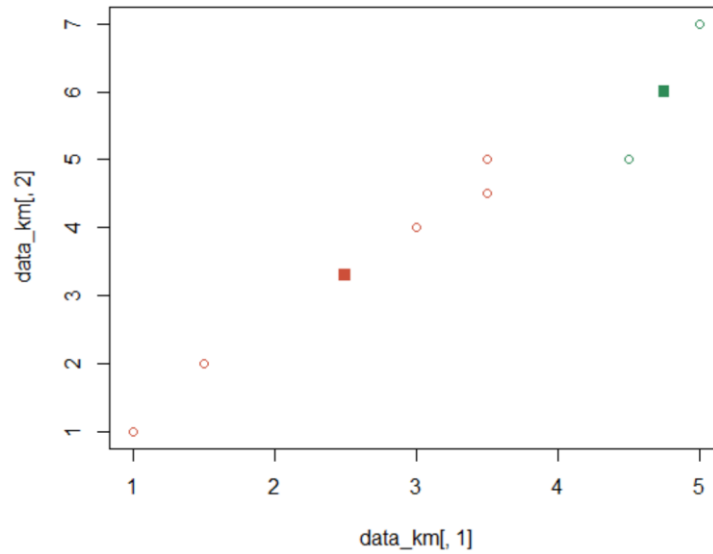


이 경우는 변화가 있으므로 다시 1번부터 반복하기로 한다.

[그림 6] [2 단계] 각 점을 가장 가까운 중심으로 할당

4. 1번의 중심값과 3번에서 선택된 중심값이 변화가 없다면 멈춘다. 만약 데이터의 변화가 있다면 다시 1번부터 반복한다.

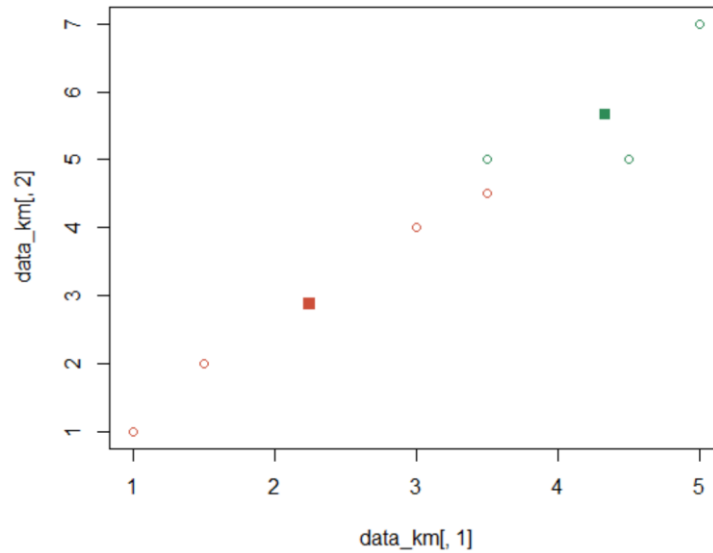
반복 2회



[그림 7] [3 단계] 각 군집의 평균을 새 중심값으로 갱신 — 반복 1 회

4. 1번의 중심값과 3번에서 선택된 중심값이 변화가 없다면 멈춘다. 만약 데이터의 변화가 있다면 다시 1번부터 반복한다.

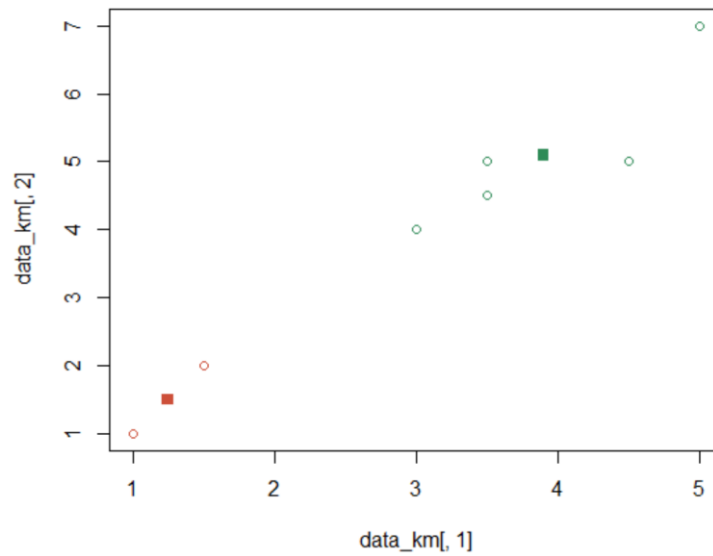
반복 3회



[그림 8] 할당과 갱신 반복 — 반복 3 회

4. 1번의 중심값과 3번에서 선택된 중심값이 변화가 없다면 멈춘다.
만약 데이터의 변화가 있다면 다시 1번부터 반복한다.

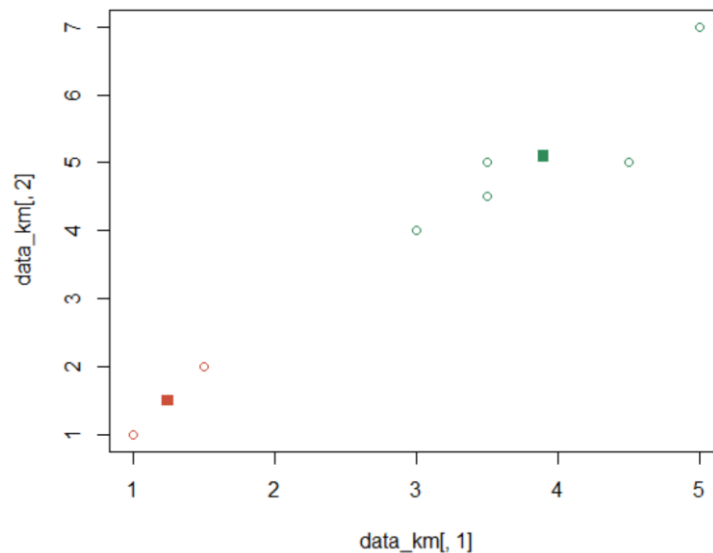
반복 5회



[그림 9] 할당과 갱신 반복 — 반복 5 회

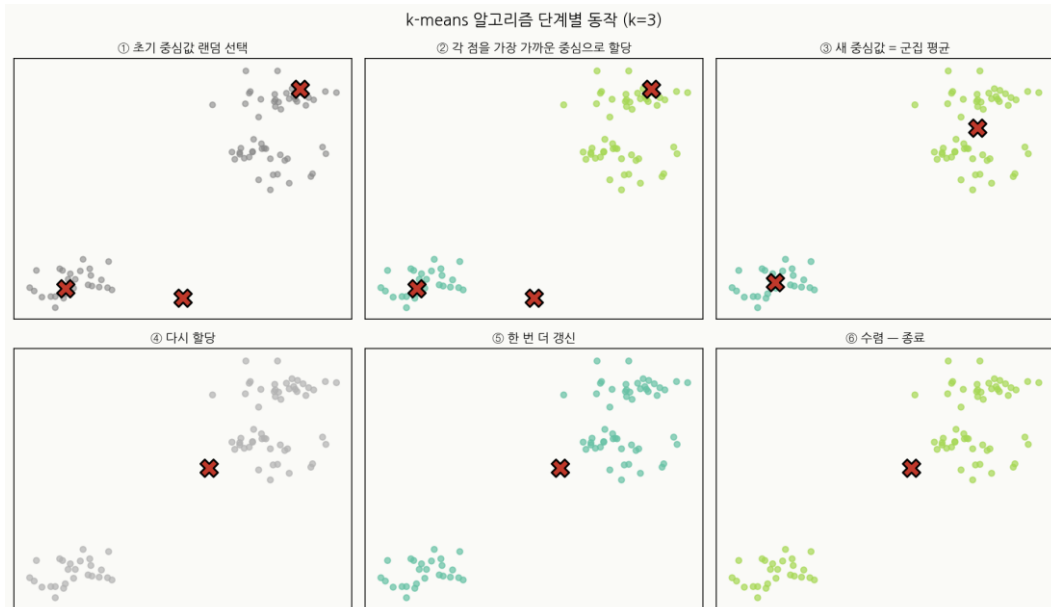
4. 1번의 중심값과 3번에서 선택된 중심값이 변화가 없다면 멈춘다.
만약 데이터의 변화가 있다면 다시 1번부터 반복한다.

반복 6회: 종료



[그림 10] 중심값 변화 없음 → 종료 (반복 6 회)

각 반복에서 중심값은 "현재 군집 멤버들의 평균" 으로 이동한다. 이 이동이 일어나면 다시 가까운 중심이 바뀌는 점이 생기고, 새 할당이 또 새 평균을 만든다. 이 "할당 → 갱신" 의 핑퐁이 더 이상 변화가 없을 때까지 반복된다.



[그림 11] k=3 데이터에 대한 K-means 단계별 동작 — 자체 생성

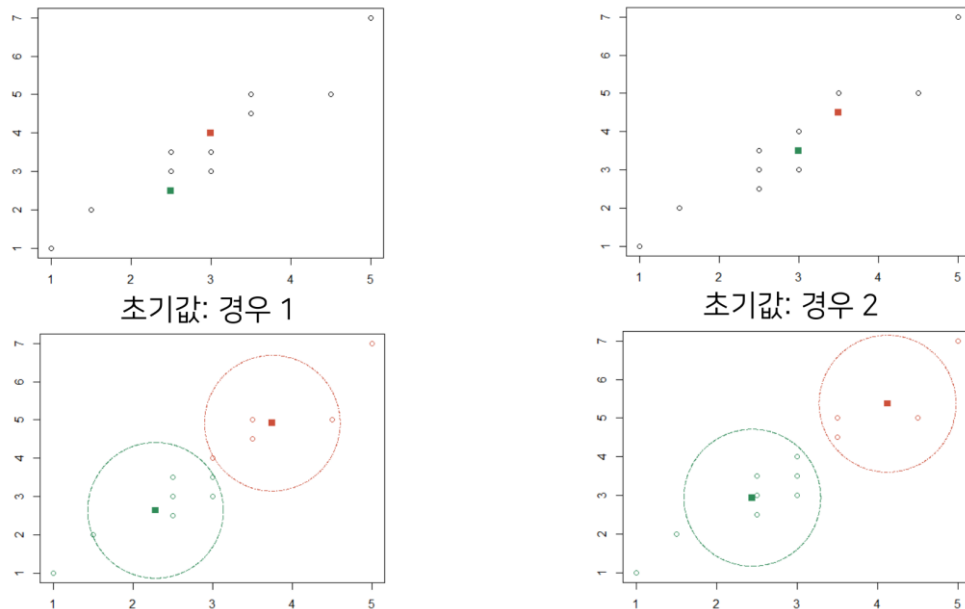
2.3 초기값 의존성

K-Means 의 약점 중 가장 잘 알려진 것은 초기 중심값에 따라 결과가 달라진다는 점이다. 운이 좋은 초기값은 적은 반복 횟수로 "좋은" 군집에 수렴하지만, 운이 나쁜 초기값은 "엉뚱한" 지역 최소에 갇힐 수 있다.

- ☑ 5. k-means 군집분석은 초기값에 따라 수렴속도가 달라질 수 있다.

[그림 12] 다른 초기값으로 시작하면 수렴 속도와 결과가 달라진다

- ☑ 6. 데이터가 정확하게 분리되기 힘든 경우(비슷한 위치에 모여 있는 경우) 초기값에 따라 분석 결과가 달라질 수 있다.



[그림 13] 초기값이 다르면 같은 데이터라도 분석 결과가 달라지는 경우

실무 대응 - `n_init` 옵션으로 여러 초기값을 시도해 그 중 WCSS 가 가장 낮은 결과를 채택한다.
 sklearn 의 KMeans 는 기본값이 `n_init=10` 이고, `k-means++` 라는 똑똑한 초기화 기법을 함께 쓴

다.

k-means++ 는 첫 중심을 랜덤하게 두고, 다음 중심은 "기존 중심에서 멀리 있는 점" 을 확률적으로 뽑는다.

이 한 줄짜리 개선이 K-means 의 안정성을 크게 끌어올렸다.

2.4 scikit-learn 으로 구현해 보기

파이썬에서는 sklearn.cluster.KMeans 한 줄로 끝난다. 단, K(n_clusters) 와 초기 중심점 배치 방식(init) 을 명시해야 한다. 기본 init 인 "k-means++" 는 초기 중심을 데이터 분포에 따라 영리하게 흩어 놓아 수렴 속도를 높여 준다.

```
# K-Means 핵심 파라미터
# class sklearn.cluster.KMeans(
#     n_clusters=8,          # 군집 개수 K - 가장 중요한 하이퍼파라미터
#     init='k-means++',      # 초기 중심점 배치 방식 ('random' 도 가능)
#     n_init=10,             # 다른 초기값으로 몇 번 시도할지
#     max_iter=300,          # 한 시도당 최대 반복 횟수
#     tol=1e-4,              # 수렴 판정 임계값
#     random_state=None)     # 재현성을 위한 시드

from sklearn.datasets import make_classification
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# 1. 데이터 생성 (3 개 군집)
X, y_true = make_classification(
    n_samples=300, n_features=2, n_informative=2, n_redundant=0,
    n_clusters_per_class=1, n_classes=3, random_state=42)

# 2. K-Means 학습 및 예측
kmeans = KMeans(n_clusters=3, n_init=10, random_state=42)
y_pred = kmeans.fit_predict(X)

# 3. 시각화
plt.figure(figsize=(6, 5))
plt.scatter(X[:, 0], X[:, 1], c=y_pred, cmap='viridis', edgecolor='k')
plt.scatter(kmeans.cluster_centers_[0], kmeans.cluster_centers_[1],
            c='red', s=100, marker='X', label='Centroids')
plt.title('KMeans Clustering (make_classification)')
plt.legend(); plt.xlabel('Feature 1'); plt.ylabel('Feature 2')
plt.grid(True); plt.tight_layout(); plt.show()
```

팁 박스 - K-Means 전에 표준화하자.

K-Means 는 거리를 쓴다. 한 변수가 0~1, 다른 변수가 0~1000 이면 거리 계산이 큰 변수에 압도된다.

StandardScaler().fit_transform(X) 로 먼저 스케일을 맞춘 뒤 KMeans 를 돌리자.

sklearn Pipeline 은 쓰지 말고 단계별로 명시적으로 호출한다.

2.5 K-Means 의 장단점

- 장점
 - 알고리즘이 단순하다. 거리 기반 할당과 평균 갱신의 두 단계뿐.

- 데이터 변환 없이 그대로 적용 가능. 표준화 정도만 신경 쓰면 된다.
- 데이터 양이 많아도 빠르다. 시간 복잡도 $O(n \cdot k \cdot t)$.

- 단점

- 거리를 기반으로 하므로 변수의 단위(scale)에 민감하다 — 표준화 필수.
- 초기 군집 수 k 를 미리 정해야 한다.
- 초기값에 따라 결과가 달라지는 비결정성.
- 범주형 변수에 직접 적용 불가. K-modes 등 변형이 필요.
- 군집이 "원형"으로 분리될 수 있는 경우에만 잘 작동. 길쭉하거나 휘어진 군집에는 약하다.
- 이상치에 약하다. 평균을 쓰므로 한두 개의 극단값이 중심을 끌어당긴다.

3. 적정 군집 수 K 의 결정

K-Means의 가장 큰 실무적 어려움은 " k 를 몇으로 둘 것인가"이다. 데이터를 보지 않고 정확한 답을 미리 알기는 어렵다. 이 장에서는 k 를 정하는 세 가지 방법 — 분석가의 직접 결정, 엘보우 기법, 실루엣 계수 — 를 본다.

3.1 분석가가 직접 결정하는 경우

데이터에 대한 사전 정보가 있거나 분석 목적이 "3개 그룹으로 나누는 것"처럼 분명하다면 분석가가 직접 k 를 정한다. 예를 들어 마케팅 부서가 "VIP, 우수, 일반" 세 단계로 고객을 나누고 싶다면 $k=3$ 으로 두면 된다. 그러나 대부분의 상황에서는 k 를 미리 정하기가 힘들기 때문에, 데이터로부터 "좋은 k "를 찾는 방법이 필요하다.

k-means 군집 분석에서 군집 수(k) 결정 방법 (1/2)

- 분석가 결정 : 데이터에 대한 **사전 정보가 있거나** 분석 목표에 의해 군집 개수가 **확실히 정해진** 경우
- 일반적으로 k-means 군집 분석을 하는 상황에서는 k 를 미리 결정하기가 힘들다.

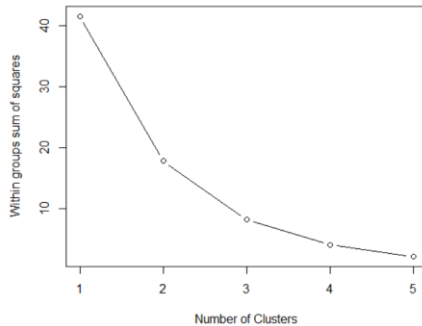
[그림 14] 군집 수 k 를 정하는 두 갈래 — 분석가 결정 vs 데이터 기반

3.2 엘보우 기법 (Elbow Method)

엘보우 기법의 발상은 단순하다. 여러 k 값에 대해 WCSS를 계산해 보고 그래프를 그린다. k 가 늘어날수록 WCSS는 반드시 감소하지만, 어느 지점부터는 감소 속도가 급격히 완만해진다. 그 "꺾이는 지점"이 사람 팔꿈치(elbow) 모양을 닮았다 해서 엘보우 기법이라 부른다. 이 꺾이는 지점의 k 를 적정 군집 수로 본다.

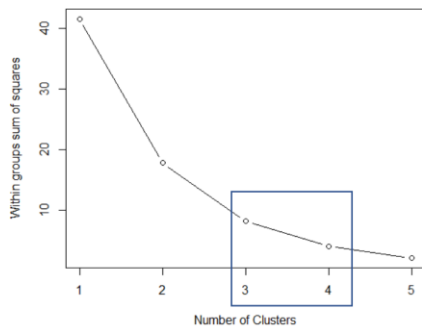
엘보우(elbow) 기법

여러 가지 k값에 대한 군집 분석 결과의 군집 내에 얼마나 밀집되어 있는가를 나타내는 통계량의 합을 기준으로 **크기의 변화가 작아지는 지점**을 적정 군집 수로 결정하는 방법.



[그림 15] 엘보우 기법의 개념 — WCSS 가 급격히 줄다 완만해지는 지점이 elbow

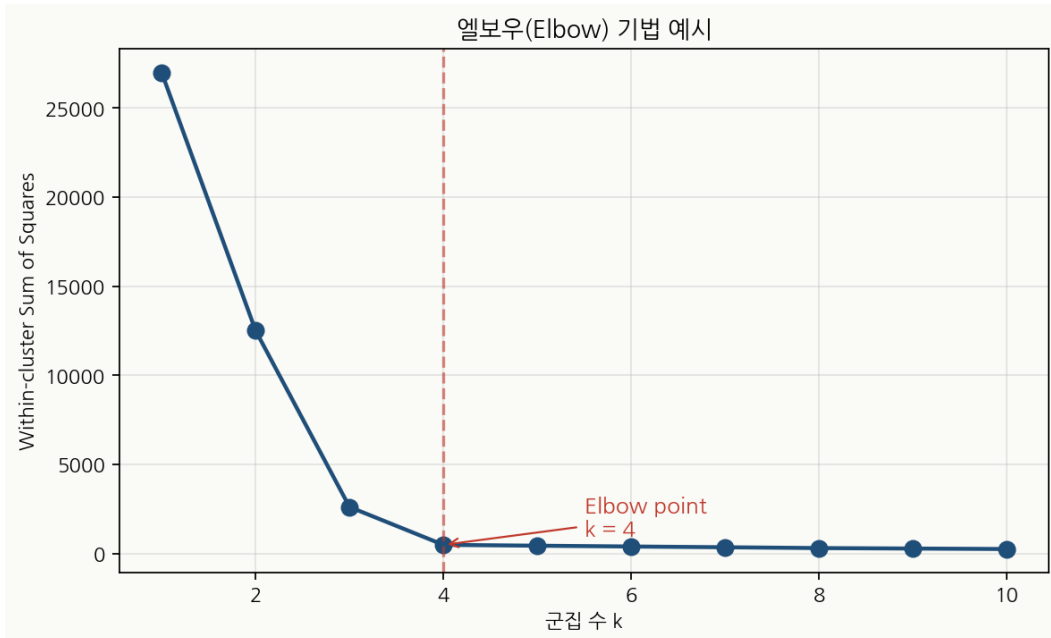
- 군집 수 k가 증가하면 within group sum of square는 항상 감소한다. 하지만 무작정 k를 늘리는 것은 좋지 않다.
- k가 증가했는데 within group sum of square가 별로 차이 나지 않는다면 (기울기가 **급격하게 완만**해진다면) 그 부분이 elbow point가 된다.



세분화 예시

좌측 예시의 경우, 군집 수를 3~4개로 하는게 적정하다고 판단됨
군집 수 결정시, 군집 수를 매우 큰 값으로 설정하면 군집 내 동질성은 향상되지만 데이터에 대한 **과적합**이 될 위험이 있음

[그림 16] 엘보우 기법의 예 — k=3~4 정도에서 꺾인다



[그림 17] make_blobs(4 centers) 데이터에 엘보우 기법을 직접 적용한 결과

```
# 엘보우 기법으로 K 찾기
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

X, _ = make_blobs(n_samples=300, centers=4, cluster_std=0.6, random_state=42)

inertias = []
K_range = range(1, 11)
for k in K_range:
    km = KMeans(n_clusters=k, n_init=10, random_state=42)
    km.fit(X)
    inertias.append(km.inertia_) # SSE 와 같은 의미

plt.plot(list(K_range), inertias, 'o-')
plt.xlabel('K'); plt.ylabel('SSE (inertia)')
plt.title('Elbow Method'); plt.grid(True); plt.tight_layout()
plt.show()
```

팁

- 엘보우가 항상 분명하게 보이지는 않는다. 데이터가 정말 명확한 군집 구조를 가질 때만 "썩이는 점"이 뚜렷이 나타난다.

곡선이 매끄러우면 엘보우 기법은 결정적인 답을 주지 못한다. 이때 실루엣 계수를 보조로 쓰면 좋다.

3.3 실루엣 계수 (Silhouette Coefficient)

실루엣 계수는 각 데이터 점에 대해 "자기 군집과의 가까움" 과 "이웃 군집과의 땀" 을 동시에 보는 지표다. 한 점 i 에 대해 $a(i)$ 를 "같은 군집 내 다른 점과의 평균 거리", $b(i)$ 를 "가장 가까운 다른 군집의 점들과의 평균 거리" 로 정의하면

실루엣 계수의 정의

a_i = 점 i 와 같은 군집 내 다른 점들 사이의 평균 거리

b_i = 점 i 와 가장 가까운 "다른 군집" 점들 사이의 평균 거리

$s_i = (b_i - a_i) / \max(a_i, b_i)$

값의 범위는 $-1 \sim +1$.

+1 에 가까울수록 - 잘 분리된 군집에 속한다.

0 부근 - 두 군집 경계에 걸쳐 있다.

-1 에 가까울수록 - 잘못된 군집에 들어가 있다.

k-means 군집 분석에서 군집 수(k) 결정 방법 (2/2)

▪ 실루엣 계수(Silhouette)

군집 내 동질적인 정도와 군집 간 이질적인 정도를 수치화한 값

▪ 실루엣 계수가 1에 가까울수록 군집화가 **잘 된 것**,

-1에 가까울수록 제대로 되지 않은 것

▪ 실루엣 계수의 정의 자체는 개별 데이터에 대해 각각 계산,

보통 실루엣 계수를 평균내서 평균 수치를 보고 군집화가 잘 됐는지 판단

[그림 19] 실루엣 계수의 정의 - 군집 내 동질성과 군집 간 이질성을 동시에 수치화

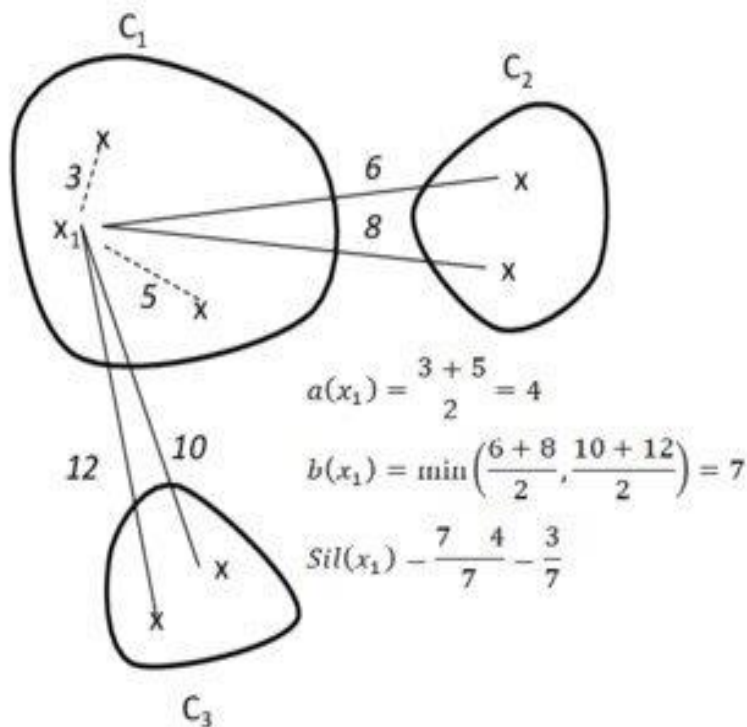
k-means 군집 분석에서 군집 수(k) 결정 방법 (2/2)

- 실루엣 계수: 군집 내 동질적인 정도와 군집 간 이질적인 정도를 수치화한 값
- 실루엣 계수가 1에 가까울수록 군집화가 잘 된 것, -1에 가까울수록 제대로 되지 않은 것
 - 개별 데이터 i 에 대해, 군집 내 동질성을 의미하는 $a(i)$ 와 군집 간 이질성을 의미하는 $b(i)$ 로 구성
 - $a(i)$: i 가 속한 군집에 있는 다른 데이터들과의 평균 거리 $\rightarrow a(i)$ 가 작을수록 좋음
 - $b(i)$: 가장 가까운 이웃 군집과의 평균 거리 $\rightarrow b(i)$ 가 클수록 좋음

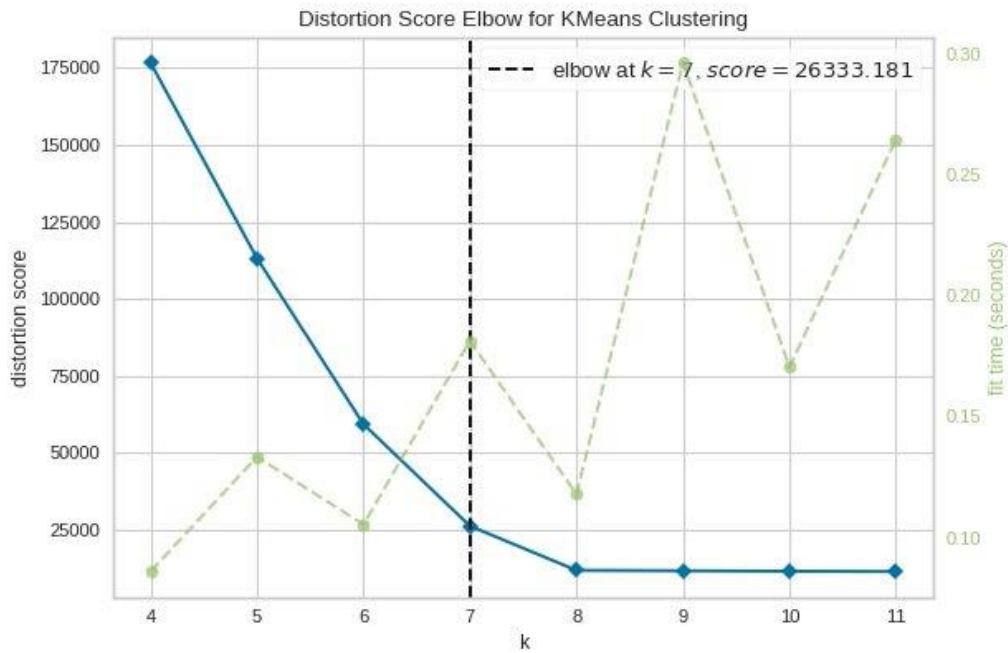
$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))} = \begin{cases} 1 - \frac{a(i)}{b(i)} & \text{if } a(i) < b(i) \\ 0 & \text{if } a(i) = b(i) \\ \frac{b(i)}{a(i)} - 1 & \text{if } a(i) > b(i) \end{cases}$$

- $a(i) \ll b(i)$ 이면 $s(i) \rightarrow 1$ 이므로 $s(i)$ 가 1에 가까우면 좋음
- 실제로는 모든 데이터에 대해 $s(i)$ 를 구해서 그 평균을 이용한다.
- 군집화에 대한 지표가 될 수 있다.

[그림 20] 실루엣 계수의 수식 정리 — $a(i)$ 와 $b(i)$ 의 관계



[그림 21] a_i 와 b_i 의 차이를 정규화해 -1~+1 범위로 만든다.

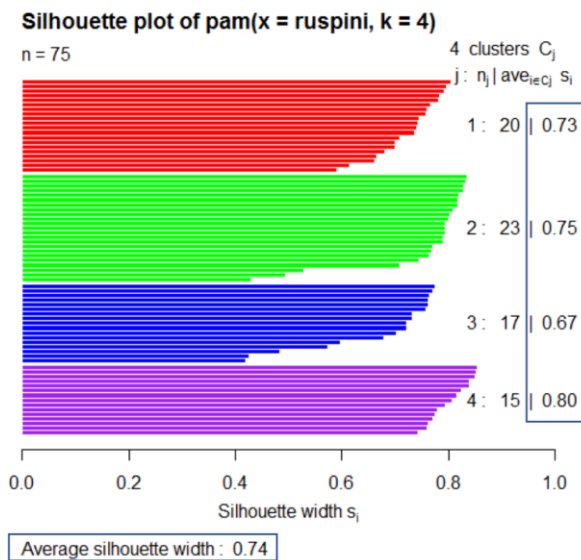


[그림 22] 실루엣 계수 산출 예시 — 군집별 실루엣 분포로 품질을 시각적으로 확인.

$s(i)$ 는 -1 부터 $+1$ 사이의 값을 가진다. 1 에 가까우면 i 는 자기 군집의 "안쪽" 에 잘 자리 잡고 있고, 0 에 가까우면 두 군집의 경계에 있으며, -1 에 가까우면 잘못 분류된 것이다. 전체 데이터의 $s(i)$ 평균을 평균 실루엣 계수라 하고, 이 값으로 k 를 비교한다.

➕ 평균 실루엣 계수가 0.5보다 크면 좋은 k 라고 판단

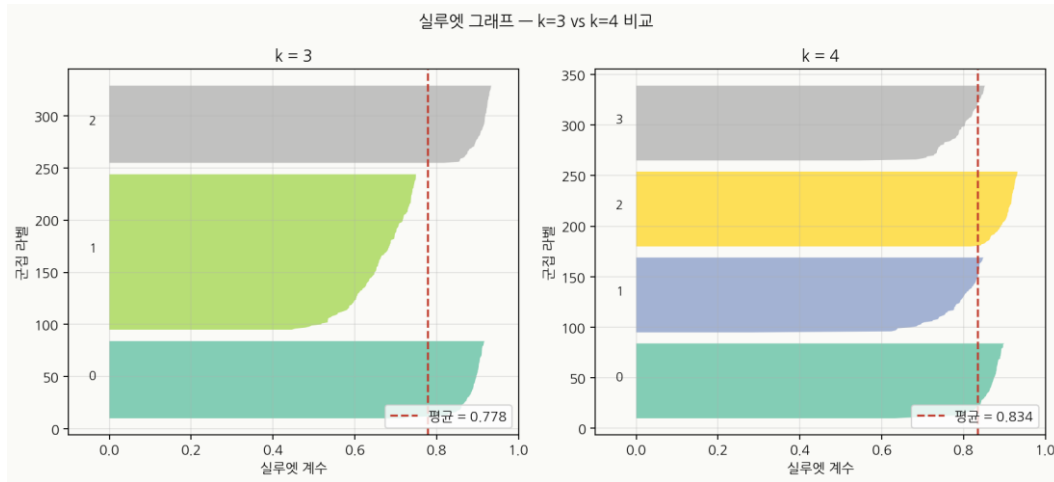
각 군집별 실루엣 계수들이 높은 경우도 적절한 k 라고 판단 가능



각 군집별 실루엣 계수 평균

데이터 전체의 실루엣 계수 평균

[그림 23] 실루엣 그래프 예 — 각 군집별 실루엣 값의 분포



[그림 24] k=3 vs k=4 의 실루엣 비교 — 본 데이터는 k=4 에서 평균이 더 높다

3.4 실루엣 평균의 해석 기준

- $-1.0 \leq \text{평균 실루엣} < 0.3$: 나쁨 — 군집 구조가 거의 없거나 잘못 분리됨.
- $0.3 \leq \text{평균 실루엣} \leq 0.5$: 보통 — 약한 구조.
- $0.5 < \text{평균 실루엣} \leq 1.0$: 좋음 — 분명한 군집 구조.
- 각 군집별 실루엣 값도 함께 보는 것이 중요. 한 군집만 매우 낮으면 그 군집이 다른 군집과 섞여 있다는 신호.

[그림 25] 실루엣 평균값의 해석 기준 정리

```
# 실루엣 계수 계산
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score, silhouette_samples
import numpy as np

X, _ = make_blobs(n_samples=300, centers=4, cluster_std=0.6, random_state=42)

kmeans = KMeans(n_clusters=4, n_init=10, random_state=42)
labels = kmeans.fit_predict(X)

# 전체 평균 실루엣
overall = silhouette_score(X, labels)
print(f'전체 평균 실루엣 계수: {overall:.4f}')

# 각 점별 실루엣 계수
sample_scores = silhouette_samples(X, labels)
for k in range(4):
    avg_k = sample_scores[labels == k].mean()
    print(f'cluster {k} 평균: {avg_k:.4f}')
```

팁 박스 — 엘보우와 실루엣을 함께 쓰자.

엘보우는 시각적 직관에 의존하고, 실루엣은 숫자로 비교한다.

두 방법이 비슷한 K 를 추천하면 그 K 를 채택해도 안전하다.

의견이 갈리면 도메인 지식이 결정한다.

4. 계층적 군집분석 (Hierarchical Clustering)

K-Means 의 "k 를 미리 정해야 한다" 라는 약점은 계층적 군집분석으로 어느 정도 우회할 수 있다. 계층적 군집은 모든 점을 각자의 군집으로 시작해 가까운 것끼리 차례로 합치거나(응집형), 전체를 하나의 군집으로 보고 점점 쪼개는(분할형) 방식을 쓴다. 결과로 얻는 트리(덴드로그램)를 보고 사후에 "여기서 자르면 k 개 군집이 되겠다" 라고 결정할 수 있다.

4.1 응집형(Agglomerative) 방식

응집형은 다음과 같이 진행된다. (1) 모든 점이 자기 자신만의 군집으로 시작한다 — n 개의 점이면 n 개의 군집. (2) 군집 간 거리가 가장 가까운 두 군집을 합친다. (3) 합쳐진 군집과 나머지 군집들 사이의 거리를 새로 계산한다. (4) 모든 점이 하나의 큰 군집으로 합쳐질 때까지 반복.

응집형의 4 단계

1. 모든 점이 자기 자신만의 군집 (n 개 점 = n 개 군집).
2. 군집 간 거리가 가장 가까운 두 군집을 합친다.
3. 합쳐진 군집과 나머지 군집들 사이의 거리를 새로 계산.
4. 모든 점이 하나의 큰 군집이 될 때까지 반복.
→ 결과는 트리(덴드로그램)로 표현된다.

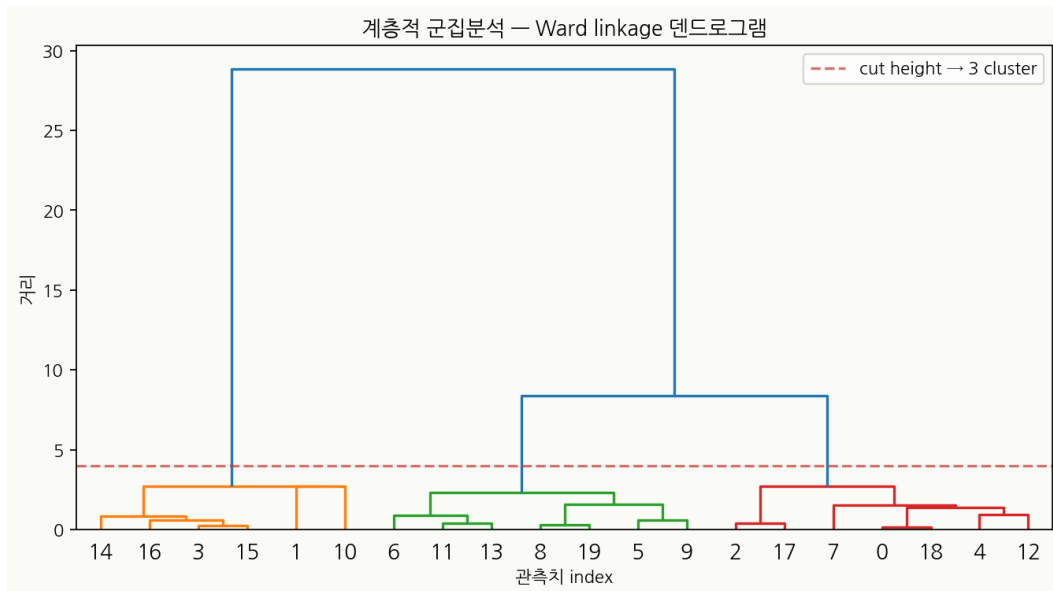
4.2 군집 간 거리 정의 (Linkage)

응집형의 핵심 선택지는 "두 군집 간 거리를 어떻게 정의할 것인가" 이다. 다음 네 가지 linkage 가 가장 자주 쓰이며, 어떤 linkage 를 쓰느냐에 따라 군집의 모양이 크게 달라진다.

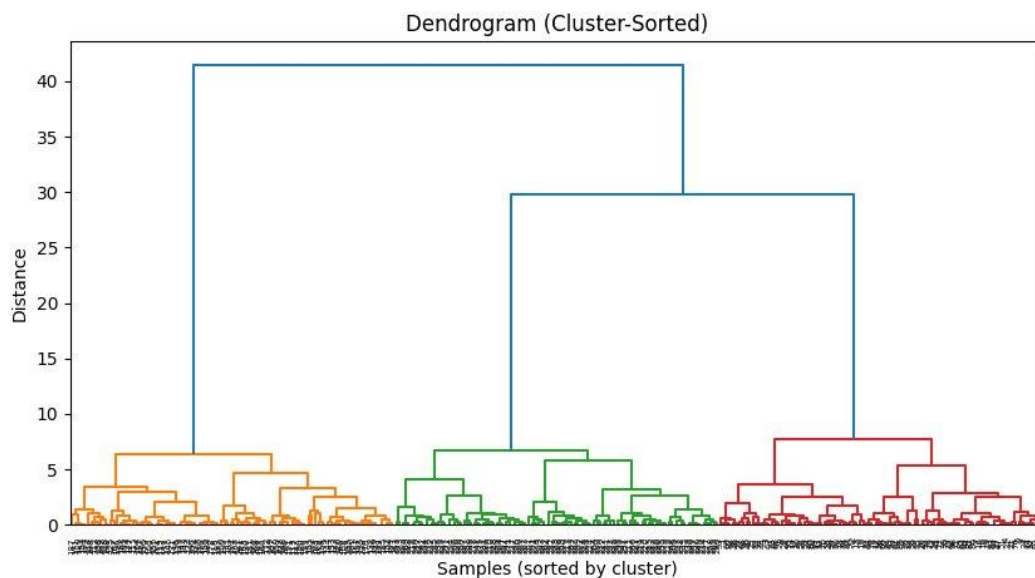
- Single linkage (최소거리) — 두 군집의 점 중 가장 가까운 두 점의 거리. 길쭉한 군집을 잘 잡지만 chaining 효과로 멀리 떨어진 군집까지 이어붙일 수 있다.
- Complete linkage (최대거리) — 가장 먼 두 점의 거리. 둥근 군집을 잘 만들지만 노이즈에 민감.
- Average linkage — 모든 쌍의 평균. single 과 complete 의 절충안.
- Ward linkage — 두 군집을 합쳤을 때 WCSS 가 가장 적게 증가하는 쌍을 선택. K-means 와 가장 비슷한 결과를 준다.

4.3 덴드로그램 읽는 법

응집형 결과를 시각화하면 트리 모양의 덴드로그램이 된다. x 축은 개별 관측치이고, y 축은 두 군집이 합쳐질 때의 거리이다. y 축의 어떤 높이에서 가로로 자르면, 그 자른 위치 아래의 가지 개수가 군집 수가 된다. 큰 점프 직전 — 즉 "가지를 합치는 비용이 갑자기 커지기 직전" — 에서 자르는 것이 좋다.



[그림 26] Ward linkage 덴드로그램 — 빨간 점선에서 자르면 3 개 군집



[그림 27] Agglomerative Clustering — 덴드로그램으로 군집 계층 구조를 확인.

4.4 scikit-learn 으로 구현

```
# Agglomerative + Dendrogram
from sklearn.cluster import AgglomerativeClustering
from sklearn.datasets import make_blobs
from scipy.cluster.hierarchy import dendrogram, linkage
import matplotlib.pyplot as plt
import numpy as np

X, _ = make_blobs(n_samples=300, centers=3, cluster_std=0.6, random_state=0)

# Agglomerative Clustering – Ward linkage
agg = AgglomerativeClustering(n_clusters=3, linkage='ward')
labels = agg.fit_predict(X)

plt.figure(figsize=(6, 4))
plt.scatter(X[:, 0], X[:, 1], c=labels, s=30)
plt.title('Agglomerative Clustering (Ward)')
```

```
plt.tight_layout(); plt.show()

# 덴드로그램 (클러스터별로 정렬)
order = np.argsort(labels)
X_sorted = X[order]
linked = linkage(X_sorted, method='ward')

plt.figure(figsize=(10, 5))
dendrogram(linked)
plt.title('Dendrogram (Cluster-Sorted)')
plt.xlabel('Samples'); plt.ylabel('Distance')
plt.tight_layout(); plt.show()
```

4.5 K-Means 와의 비교

- 사전 k 결정 — K-Means: 필요. 계층적: 불필요(사후 cut height 로 결정).
- 계산 비용 — K-Means: $O(n \cdot k \cdot t)$. 계층적: $O(n^2 \cdot \log n)$ 또는 $O(n^3)$ — 데이터가 크면 비싸다.
- 결과의 결정성 — K-Means: 초기값에 의존. 계층적: 결정적(같은 데이터·linkage 면 항상 같은 트리).
- 시각화 — K-Means: 점 색깔로만. 계층적: 덴드로그램이라는 추가 자료를 얻는다.
- 확장성 — K-Means: 대용량 데이터에 강함. 계층적: n 이 수만 개를 넘으면 부담.

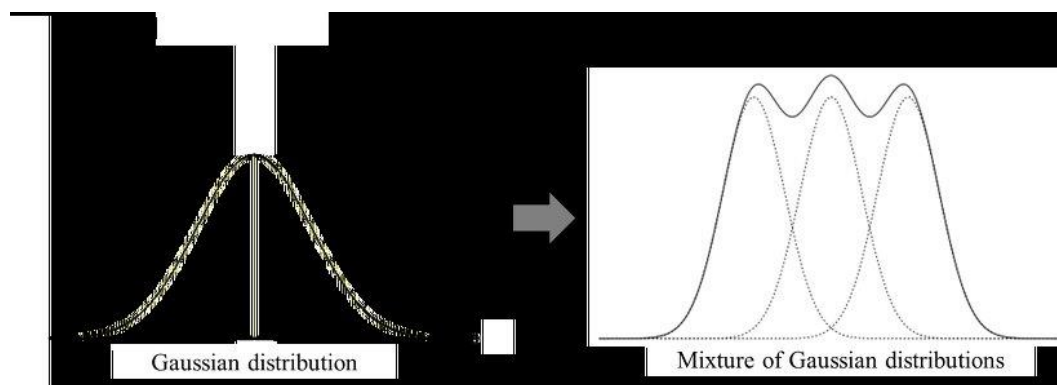
실무 가이드 – 데이터가 작고 군집의 자연스러운 계층 구조가 궁금하면 계층적.

데이터가 크고 군집 수를 정해놓고 빠르게 결과만 보고 싶으면 K-Means.

두 방법을 함께 써서 "계층적으로 덴드로그램을 보고 k 를 결정 → K-Means 로 빠르게 적합" 하는 패턴도 흔하다.

5. Mean Shift Clustering

K-Means 가 "중심에서 가까운 점들" 을 묶는다면, Mean Shift 는 "점들이 뭉쳐있는 곳" 을 찾아간다. 확률 밀도가 높은 봉우리(mode) 를 향해 중심을 옮겨 가며 군집을 형성하는 방식이다. 컴퓨터 비전에서 영상 분할, 객체 추적에 많이 쓰인다.



[그림 28] 대역폭 안에서 가장 밀도가 높은 방향으로 중심이 이동한다.

Mean Shift 의 특징

- 군집 개수 K 를 사전에 지정할 필요가 없다 – 대역폭(bandwidth) 이 결과를 결정한다.
- 데이터의 분포 형태를 가정하지 않아 유연하다.

- 이상치에 상대적으로 강하다.
- 대역폭이 크면 군집이 적게, 작으면 많이 잡힌다. `estimate_bandwidth` 로 적절한 값을 추정 가능.

```
# Mean Shift 구현
from sklearn.cluster import MeanShift, estimate_bandwidth
from sklearn.datasets import make_blobs
import matplotlib.pyplot as plt

X, _ = make_blobs(n_samples=500, centers=2, cluster_std=0.6, random_state=0)

# 대역폭 추정 (quantile 이 클수록 대역폭도 크다)
best_bw = estimate_bandwidth(X, quantile=0.2)

# Mean Shift 학습
ms = MeanShift(bandwidth=best_bw)
ms.fit(X)
labels = ms.labels_
centers = ms.cluster_centers_

# 시각화
plt.scatter(X[:, 0], X[:, 1], c=labels, s=30)
plt.scatter(centers[:, 0], centers[:, 1], c='red', marker='x', s=100)
plt.title('Mean Shift Clustering')
plt.tight_layout(); plt.show()
```

6. Gaussian Mixture Model (GMM)

K-Means 는 점을 한 군집에 "딱" 할당한다. 그런데 현실에서는 한 점이 여러 군집에 속할 가능성이 있다 — 두 분포가 겹치는 경계에서 더욱 그렇다. GMM 은 이런 모호함을 확률로 표현한다. "이 점은 클러스터 1 에 60%, 클러스터 2 에 40% 속한다" 같은 답을 준다.

6.1 핵심 아이디어 — 여러 가우시안의 혼합

GMM 의 가정은 단순하다. 데이터가 K 개의 가우시안 분포의 혼합으로 만들어졌다고 본다. 각 가우시안은 평균 μ_k 와 공분산 Σ_k 를 가진다. K 개의 가우시안 모수를 추정하면 군집이 정해진다.

```

## k-means와 비교하기
kmeans = KMeans(n_clusters = 3, init = "k-means++", max_iter = 300, random_state = 0).fit(iris.data)
kmeans_cluster_labels = kmeans.predict(iris.data)
iris_df["kmeans_cluster"] = kmeans_cluster_labels
iris_result_km = iris_df.groupby(["target"])[kmeans_cluster].value_counts()
print(iris_result_km)

target  kmeans_cluster
0        1              50
1        2              48
        0               2
2        0              36
        2              14
Name: kmeans_cluster, dtype: int64

[21] iris_df["gmm_cluster"] = gmm_cluster_labels
iris_df["target"] = iris.target
iris_result = iris_df.groupby(["target"])[gmm_cluster].value_counts()
print(iris_result)

target  gmm_cluster
0        0              50
1        1              45
        2               5
2        2              50
Name: gmm_cluster, dtype: int64

2번 타입의 target을 잘 구분해내지 못했음.

```

[그림 29] K=3 일 때의 Gaussian Mixture — 세 개의 가우시안 분포가 겹쳐 데이터를 설명한다.

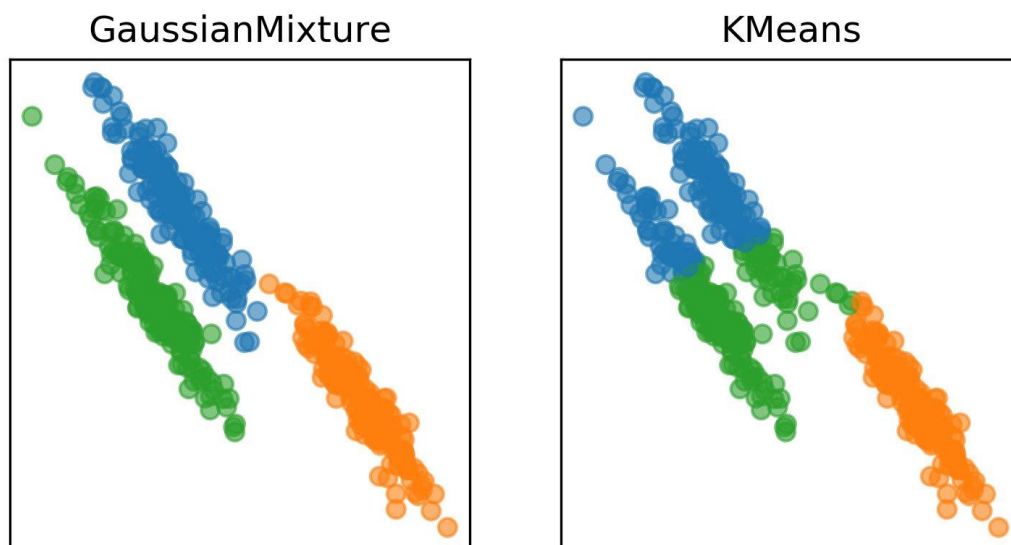
GMM 의 두 가지 추정

1. 각 가우시안의 모수 (μ_k, Σ_k) — 즉 군집의 위치와 모양.
2. 각 점이 어느 가우시안에서 나왔을 확률 — 즉 소속 확률.

이 둘을 번갈아 추정하는 알고리즘이 EM (Expectation-Maximization) 알고리즘이다.

6.2 K-Means 와의 비교

iris 데이터에 K-Means 와 GMM 을 둘 다 돌려 보면, K-Means 가 GMM 보다 살짝 떨어지는 경우가 많다. 이유는 알고리즘 자체의 우열이 아니라 데이터의 형태 때문이다 — iris 의 세 군집은 완벽한 구형이 아니라 길쭉한 타원형에 가깝다. K-Means 는 구형 가정을 깔고 있어서 약간 헛갈리고, GMM 은 공분산을 추정하니 타원도 잘 다룬다.



```
# GMM 구현
from sklearn.datasets import make_blobs
from sklearn.mixture import GaussianMixture
import matplotlib.pyplot as plt

X, _ = make_blobs(n_samples=300, centers=3, cluster_std=1.0, random_state=42)

gmm = GaussianMixture(n_components=3, covariance_type='full', random_state=42)
gmm.fit(X)
labels = gmm.predict(X)

# 각 점의 소속 확률
proba = gmm.predict_proba(X)
print(f'첫 5 개 점의 소속 확률:')
print(proba[:5].round(3))

# 시각화
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis', edgecolor='k')
plt.title('Gaussian Mixture Model Clustering')
plt.tight_layout(); plt.show()
```

팁 박스 – GMM 은 K-Means 보다 느리다.

EM 알고리즘으로 모수를 추정하느라 K-Means 보다 수렴이 느리다.

대신 타원형 군집과 소속 확률이 필요한 문제에서는 GMM 이 더 적합하다.

covariance_type 을 spherical/diag/tied/full 중에 골라 자유도를 조절할 수 있다.

7. DBSCAN (Density-Based Spatial Clustering)

DBSCAN 은 1996 년에 제안된 밀도 기반 군집화의 대표 주자다. K-Means · GMM 이 잘 못 다루는 "기하학적으로 복잡한 데이터" — 초승달, 나선, 길게 뻗은 모양 — 에 강하다. 게다가 이상치를 자동으로 분리해 주니, 데이터에 잡음이 많을 때 특히 유용하다.

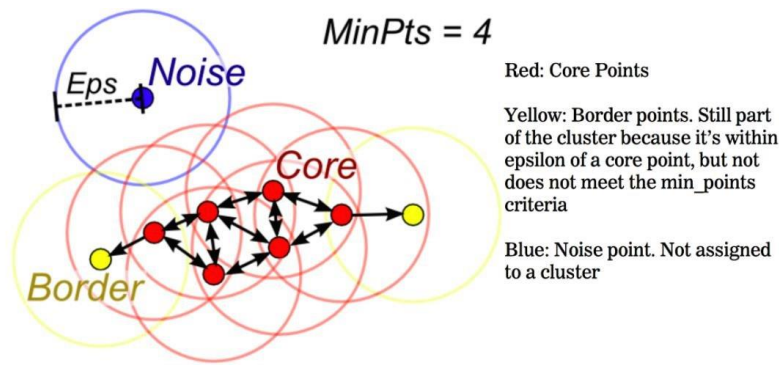
7.1 두 개의 핵심 파라미터

DBSCAN 의 두 파라미터

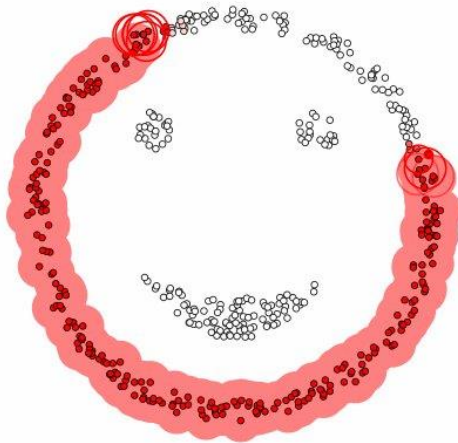
- eps (입실론) – 한 점을 중심으로 한 원형 영역의 반지름.
- min_samples – 그 영역 안에 들어와야 할 최소 이웃 수.

두 값을 통해 점은 세 종류로 나뉜다:

- 핵심 포인트 (Core) – 자신의 eps 영역 안에 min_samples 이상의 이웃이 있는 점.
- 경계 포인트 (Border) – 핵심 포인트는 아니지만 핵심 포인트의 이웃인 점.
- 잡음 포인트 (Noise) – 어느 핵심 포인트의 이웃도 아닌 점. 이상치로 분류.



[그림 31] DBSCAN 의 점 분류 — 핵심(Core)·경계(Border)·잡음(Noise) 세 종류.



[그림 32] DBSCAN 알고리즘 — 핵심 포인트가 이웃의 이웃을 통해 군집을 확장한다.

7.2 알고리즘의 진행

DBSCAN 은 임의의 점에서 출발해 그 점이 핵심 포인트이면 같은 군집을 부여하고, 핵심 포인트의 이웃의 이웃까지 전염병처럼 군집을 확장한다. 이웃이 부족해 핵심이 못 되는 경계 포인트는 그 자리에서 멈추고, 어디에도 못 닿은 점은 잡음으로 남는다.

```
# DBSCAN 구현
from sklearn.cluster import DBSCAN
from sklearn.datasets import make_blobs
import matplotlib.pyplot as plt

X, _ = make_blobs(n_samples=300, centers=4, cluster_std=0.6, random_state=42)

db = DBSCAN(eps=1.0, min_samples=5, metric='euclidean')
labels = db.fit_predict(X)
# labels == -1 이면 잡음(이상치)

n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
n_noise = list(labels).count(-1)
print(f'군집 개수: {n_clusters}, 잡음 점 개수: {n_noise}')

# 시각화 - 잡음은 검은색으로
plt.figure(figsize=(6, 5))
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis', edgecolor='k')
plt.title('DBSCAN Clustering (make_blobs)')
plt.tight_layout(); plt.show()
```


DBSCAN 의 결과 labels 에서 -1 은 "잡음(noise)" 을 뜻한다. 이 값을 가진 점들은 어떤 군집에도 속하지 못한 이상치다. 이 점만 따로 빼서 보면 자연스러운 이상치 탐지가 된다.

팁 박스 – eps 와 min_samples 의 감 잡기.

경험상 min_samples 는 차원 수의 두 배 정도 (2D 데이터면 4~5 정도).

eps 는 k-거리 그래프(k-distance graph) 로 정한다 – 각 점에서 k 번째 이웃까지의 거리를 오름차순으로 정렬해 그렸을 때 꺾이는 지점이 적절한 eps 값이다.

8. 그 외 알고리즘들

강의 시간에 다 다룰 수는 없지만 알아 두면 좋은 알고리즘들이다. 모두 scikit-learn 에 구현돼 있고 호출 방식도 비슷하다 — n_clusters 또는 eps 같은 파라미터만 같아 끼우면 된다.

8.1 BIRCH (Balanced Iterative Reducing and Clustering using Hierarchies)

BIRCH 는 대용량 데이터에 강하다. 데이터를 한 번 훑으면서 작은 요약(little summary) 을 만들고, 그 요약끼리 다시 군집화한다. 메모리에 다 안 들어가는 데이터에 적합하다. 단점은 수치형 데이터만 다룬다는 점.

```
# BIRCH 구현
from sklearn.cluster import Birch
from sklearn.datasets import make_blobs
import matplotlib.pyplot as plt

X, _ = make_blobs(n_samples=300, centers=4, cluster_std=0.6, random_state=0)

birch = Birch(n_clusters=4)
labels = birch.fit_predict(X)

plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis', edgecolor='k')
plt.title('BIRCH Clustering')
plt.tight_layout(); plt.show()
```

8.2 Affinity Propagation

Affinity Propagation 은 "각 점이 서로에게 메시지를 보내며 자신이 대표가 될지 다른 점을 대표로 추대할지 결정한다" 는 발상이다. 군집 개수를 사전에 지정할 필요가 없다. 컴퓨터 비전, 자연어 처리에서 적합한 대표 샘플(exemplar) 을 뽑을 때 쓴다.

```
# Affinity Propagation 구현
from sklearn.cluster import AffinityPropagation
from sklearn.datasets import make_blobs
import matplotlib.pyplot as plt

X, _ = make_blobs(n_samples=300, centers=4, cluster_std=0.6, random_state=42)

ap = AffinityPropagation(damping=0.9, preference=-50, random_state=0)
labels = ap.fit_predict(X)

plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis', edgecolor='k')
plt.title('Affinity Propagation Clustering')
plt.tight_layout(); plt.show()
```

8.3 OPTICS (Ordering Points to Identify Clustering Structure)

OPTICS 는 DBSCAN 의 사촌이다. DBSCAN 이 모든 군집에 같은 `eps` 를 강제하는 데 반해 OPTICS 는 밀도가 서로 다른 군집을 동시에 잡을 수 있다. 더 느리지만 더 유연하다.

```
# OPTICS 구현
from sklearn.cluster import OPTICS
from sklearn.datasets import make_blobs
import matplotlib.pyplot as plt

X, _ = make_blobs(n_samples=300, centers=3, cluster_std=0.5, random_state=0)

optics = OPTICS(min_samples=15)
optics.fit(X)
labels = optics.labels_

plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis', s=30)
plt.title('OPTICS Clustering')
plt.tight_layout(); plt.show()
```

9. Iris 데이터로 코드 실습

실제 데이터로 K-Means 의 전 과정 — 표준화·학습·평가 — 을 따라가 본다. 학습 데이터는 머신러닝에서 가장 유명한 Iris 데이터셋이다. 3 종의 붓꽃 데이터 150 개로, 군집 수 3 을 미리 알고 있어 결과 검증이 쉽다. 표준화는 `StandardScaler` 로 명시적으로 처리한다(Pipeline 미사용).

9.1 표준화 → 학습 → 평가

```
# Python - K-means: 학습과 평가 (Pipeline 미사용)
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

iris = load_iris()
X = iris.data

# Step 1: 표준화
scaler = StandardScaler()
X_s = scaler.fit_transform(X)

# Step 2: K-means 학습 (k=3)
km = KMeans(n_clusters=3, n_init=20, random_state=42)
km.fit(X_s)
labels = km.labels_

# Step 3: 평가
print('inertia (WCSS):', round(km.inertia_, 2))
print('silhouette  :', round(silhouette_score(X_s, labels), 3))
```

9.2 엘보우 기법으로 k 탐색

```
# Python - 엘보우 기법
import numpy as np
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

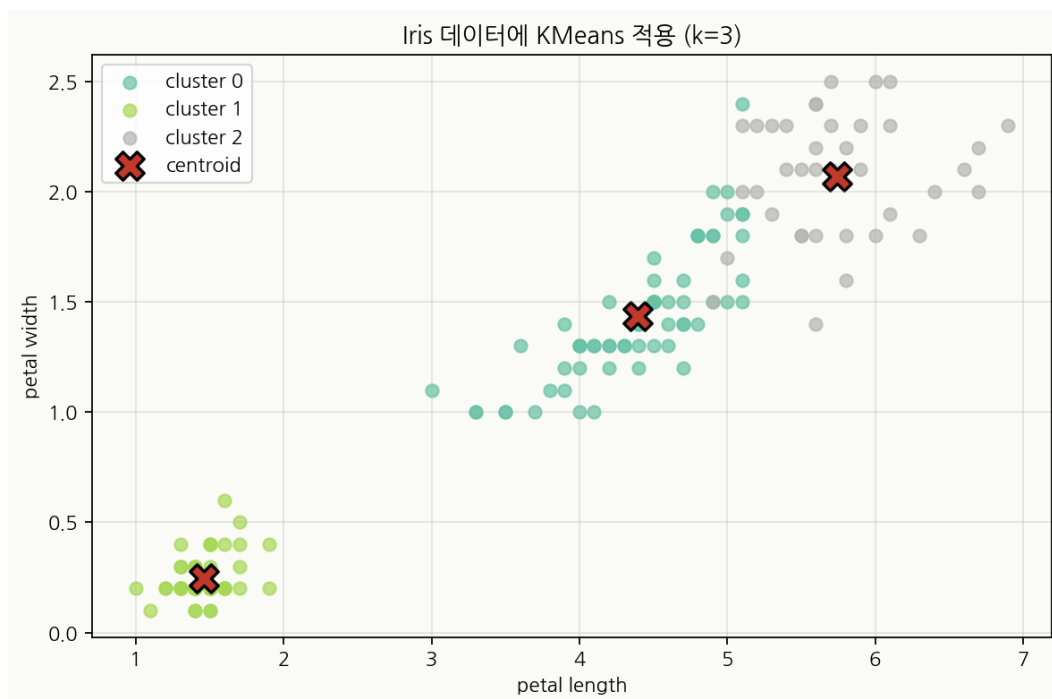
Ks = range(1, 11)
wcss = []
for k in Ks:
    m = KMeans(n_clusters=k, n_init=10, random_state=0).fit(X_s)
    wcss.append(m.inertia_)

plt.plot(list(Ks), wcss, 'o-')
plt.xlabel('k'); plt.ylabel('WCSS')
plt.title('Elbow Method')
plt.show()
```

9.3 실루엣 계수로 k 비교

```
# Python - 실루엣 계수로 k 비교
from sklearn.metrics import silhouette_score

for k in range(2, 8):
    m = KMeans(n_clusters=k, n_init=10, random_state=0).fit(X_s)
    s = silhouette_score(X_s, m.labels_)
    print(f'k={k}: silhouette={s:.3f}')
```



[그림 33] Iris 데이터에 K-means(k=3)를 적용한 결과 — petal length vs petal width

9.4 결과 비교 시 체크포인트

- 표준화를 했는가 — 안 했다면 큰 단위 변수가 거리를 지배.
- n_init 가 충분한가 — 1 회만 돌리면 운에 좌우. 보통 10~20.
- k 의 후보를 여러 개 시도했는가 — 엘보우와 실루엣을 함께 본다.

- K-Means 와 계층적 군집의 라벨이 일치하는가 — adjusted_rand_score 로 정량 비교.
- 군집 라벨은 의미가 없다 — "cluster 0" 과 "cluster 1" 은 단순 번호. 결과 해석 시 라벨 번호에 의미를 부여하지 말 것.

팁 – sklearn 의 KMeans 라벨과 scipy fcluster 라벨은 "같은 군집이라도 번호가 다를 수 있다".
두 결과의 일치도를 보고 싶으면 sklearn.metrics.adjusted_rand_score 를 쓴다.
이 지표는 라벨 번호의 순서를 무시하고 "같은 군집 쌍" 의 일치도만 본다.

10. 알고리즘 비교 요약

여기까지 본 알고리즘들을 한자리에 모아 비교해 보자. 데이터의 모양과 목적이 알고리즘을 결정한다.

알고리즘	부류	강점	약점
K-Means	거리 기반	빠르고 단순, 큰 데이터 OK	구형 가정, K 지정 필요, 초기값 의존
Agglomerative (계층적)	계층 기반	K 사후 결정, 덴드로그램 시각화, 결정적	$O(n^2 \sim n^3)$, 대용량 부담
Mean Shift	밀도/모드 기반	K 자동, 이상치 강함	대역폭 민감, 느림
GMM	분포 기반	타원형 OK, 소속 확률	EM 느림, 분포 가정
DBSCAN	밀도 기반	임의 모양, 이상치 분리, K 자동	eps 설정 어려움
BIRCH	계층 기반	대용량 메모리 효율	수치형만, 구형 가정
Affinity Prop.	메시지 전파	K 자동, 대표 샘플	$O(n^2)$ 메모리, 느림
OPTICS	밀도 기반	밀도 다른 군집 OK	DBSCAN 보다 느림

[그림 34] 군집화 알고리즘 비교표 — 데이터 형태에 따라 선택이 달라진다.

실무에서는 한 알고리즘만 쓰지 않는다. 보통 K-Means 로 빠르게 첫 그림을 그려 보고, 결과가 마음에 안 들면 계층적 군집으로 덴드로그램을 보거나 DBSCAN·GMM 을 추가로 돌려 비교한다. 도메인 지식과 시각화가 최종 판단의 기준이 된다.

11. 정리 및 요약

여기까지가 군집분석의 전부다. 라벨이 없는 데이터에서 구조를 발견하는 일은 답이 정해진 문제가 아니므로, 여러 알고리즘을 시도해 보고 데이터의 모양에 맞는 도구를 고르는 안목이 핵심이다.

11.1 핵심 개념 한 줄 정리

- 군집분석 — 라벨 없는 데이터를 비슷한 것끼리 묶는 비지도학습. 정답이 없으므로 평가가 어렵다.
- 동질성과 이질성 — 좋은 군집화 = 같은 군집 내부는 가깝고(동질성), 다른 군집 사이는 멀다(이질성).
- 네 가지 부류 — 거리·밀도·분포·계층 기반. 데이터 모양에 따라 적합한 부류가 다르다.
- K-Means — k 개 중심값을 두고 "할당 → 갱신" 을 반복해 WCSS 를 최소화. 빠르고 단순.
- 초기값 의존성 — $n_{init}=10\sim20$ 과 k-means++ 로 완화. 운에 맡기지 말 것.
- 엘보우 기법 — WCSS 곡선이 꺾이는 지점을 적정 k 로. 곡선이 매끄러우면 결정적이지 않다.
- 실루엣 계수 — $-1 \sim +1$ 범위. 평균이 0.5 초과면 좋은 군집, 0.3 미만이면 군집 구조가 약하다.
- 계층적 군집 — k 를 사후에 결정 가능. 덴드로그램이 추가 정보. 결정적이라 보고서에 안전.
- Mean Shift — 밀도가 높은 봉우리로 중심을 옮긴다. K 자동, 대역폭이 핵심 파라미터.
- GMM — 데이터를 K 개 가우시안의 혼합으로 모델링. 타원형 군집과 소속 확률을 다룬다.
- DBSCAN — 밀도 기반. eps 와 min_samples 로 핵심/경계/잡음을 구분. 이상치 탐지에도 유용.
- BIRCH·Affinity Propagation·OPTICS — 각자 특수한 상황에 강점이 있다.
- 전처리 — K-Means·GMM 같은 거리 기반은 반드시 표준화(StandardScaler) 후 사용.

11.2 알고리즘 선택 가이드

데이터 크기 — $n > 10000$ 이면 K-Means 가 사실상 강제. 계층적은 비현실적.

k 사전 결정 가능성 — 가능하면 K-Means. 불가능하면 계층적 또는 엘보우+ 실루엣 조합.

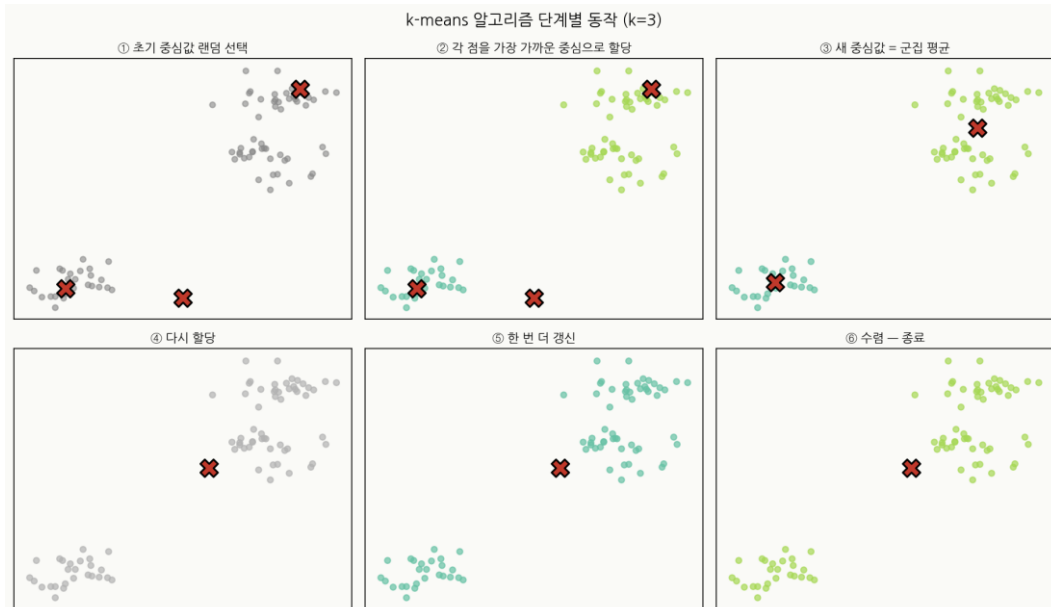
군집 형태 — 둥근 군집은 K-Means/Ward 가 강함. 비선형/길쭉한 군집은 DBSCAN 같은 밀도 기반이 더 좋다.

결과 해석의 안정성 — 계층적은 결정적, K-Means 는 비결정적. 보고서용은 계층적이 더 안전.

소속의 모호함 — 한 점이 여러 군집에 걸쳐 있을 가능성이 크면 GMM 의 소속 확률을 보자.

11.3 흔한 실수

- 표준화를 잊는다 — 단위가 큰 변수 하나가 결과를 지배.
- n_{init} 를 1 로 둔다 — 운에 따라 다른 답이 나옴.
- k 를 한 번 정해 놓고 끝낸다 — 엘보우와 실루엣을 함께 봐야 함.
- 군집 번호에 의미를 부여한다 — cluster 0 과 cluster 2 사이에 "순서" 가 없다.
- 범주형 변수에 그대로 K-Means 적용 — 거리 정의가 무너진다. K-modes 또는 Gower 거리.
- 이상치를 무시하고 K-Means — 평균이 끌려가 중심이 왜곡됨. DBSCAN 으로 먼저 이상치 제거 고려.



[그림 35] K-Means 의 동작을 한 장으로 — 본 강의의 핵심 그림

12. 과제

아래 5 문제는 본 강의의 모든 챕터를 종합적으로 점검하는 문제다. 자기 손으로 풀어 보고, 코드 문제는 직접 실행해 결과를 확인하자.

1. 군집분석과 분류(classification) 의 차이를 (1) 학습 형태, (2) 사전 정보의 유무, (3) 평가 지표의 세 측면에서 표 형식으로 비교하라. "동질성" 과 "이질성" 이 군집분석의 평가에서 어떤 역할을 하는지도 함께 설명하라.
2. K-Means 의 5 단계 알고리즘을 한 줄씩 적고, WCSS(Within-Cluster Sum of Squares) 가 한 번의 반복마다 단조 감소하는 이유를 설명하라. 그리고 "지역 최소" 문제를 완화하는 두 가지 실무 기법(`n_init`, `k-means++`) 을 1~2 줄씩 설명하라.
3. `sklearn` 의 `make_moons` 데이터셋(초승달 두 개)에 대해 K-Means 와 DBSCAN 을 각각 적용하고 결과를 시각화해 비교하라. 어느 쪽이 더 자연스럽게 두 초승달을 분리하며, 그 이유를 "거리 기반" 이라는 키워드를 써서 설명하라.
4. 어떤 데이터의 평균 실루엣 계수가 K=3 에서 0.62, K=4 에서 0.58, K=5 에서 0.43 으로 나왔다. 엘보우 그래프에서는 K=4 에서 살짝 꺾이는 모습이다. 본 강의의 실루엣 해석 기준(0.3 / 0.5) 을 근거로 어느 K 를 선택할지 결정하고 그 근거를 설명하라.

과제 해답

1 번 해답

학습 형태 — 분류는 (X, y) 쌍이 주어진 지도학습이고 군집분석은 X 만 있는 비지도학습이다.
사전 정보 — 분류는 범주의 개수와 각 데이터의 범주가 모두 알려져 있지만, 군집분석은 군집의 개수와 속성이 모두 미지수다. 평가 지표 — 분류는 정확도·F1·AUC 처럼 정답과의 일치도로 평가하고, 군집분석은 정답이 없으므로 실루엣 계수·엘보우·WCSS 같은 내부 지표를 쓴다. "동질성" 은 같은 군집 내 점들이 얼마나 가까운지(군집 내 분산이 작은가)를, "이질성" 은 서로 다른 군집의 점들이 얼마나 멀리 떨어져 있는지를 측정한다. 좋은 군집화는 동질성과 이질성이 모두 높은 상태이며, 실루엣 계수는 이 두 양을 한 숫자로 요약한다.

2 번 해답

K-Means 의 5 단계: (1) k 결정 및 초기 중심값 배치, (2) 각 점을 가장 가까운 중심에 할당, (3) 각 군집의 평균을 새 중심값으로 갱신, (4) 할당이 바뀌었으면 (2)로 복귀, (5) 변화가 없으면 종료. 한 번의 반복마다 WCSS 가 단조 감소하는 이유는 "할당 단계" 와 "갱신 단계" 모두 WCSS 를 줄이는 방향으로 동작하기 때문이다 — 할당 단계는 각 점을 가장 가까운 중심에 두므로 거리 제곱합이 줄고, 갱신 단계는 군집의 평균이 WCSS 를 최소화하는 위치이므로 또 줄어든다. 지역 최소 회피: n_init=10 이상으로 여러 초기값을 시도해 가장 낮은 WCSS 결과를 채택하고, k-means++ 로 첫 중심을 데이터 분포에 맞춰 영리하게 흠어 놓는다.

3 번 해답

```
from sklearn.datasets import make_moons
from sklearn.cluster import KMeans, DBSCAN
import matplotlib.pyplot as plt

X, _ = make_moons(n_samples=300, noise=0.05, random_state=0)

km_labels = KMeans(n_clusters=2, n_init=10, random_state=0).fit_predict(X)
db_labels = DBSCAN(eps=0.2, min_samples=5).fit_predict(X)

fig, axes = plt.subplots(1, 2, figsize=(10, 4))
axes[0].scatter(X[:, 0], X[:, 1], c=km_labels, cmap='viridis')
axes[0].set_title('KMeans on Moons')
axes[1].scatter(X[:, 0], X[:, 1], c=db_labels, cmap='viridis')
axes[1].set_title('DBSCAN on Moons')
plt.tight_layout(); plt.show()
```

DBSCAN 이 두 초승달을 자연스럽게 분리한다. K-Means 는 거리 기반으로 중심점에서 가까운 점을 모으는 구형 가정을 깔고 있어서 초승달을 가로지르는 직선 경계로 갈라 버린다. 반면 DBSCAN 은 밀도가 이어진 점들을 따라가므로 초승달 모양 그대로 군집을 형성한다. 거리만 보는 알고리즘에는 "띠 모양으로 휘어진 군집" 의 두 끝이 서로 멀어 보이기 때문에 한 군집으로 묶기 어렵다.

4 번 해답

K=3 을 선택한다. 평균 실루엣 0.62 는 본 강의의 해석 기준(0.5 초과 = 좋음) 에 부합한다. K=4 의 0.58 도 "좋음" 구간이지만 K=3 보다 낮고, K=5 의 0.43 은 "보통" 구간으로 떨어진다. 엘보우가 K=4 를 약간 추천하긴 하지만 그 꺾임이 약하고, 실루엣이 동시에 떨어진다는 점은 K=4 가 무리한 분할일 가능성을 시사한다. 두 지표가 같릴 때는 더 단순한 군집화(K 가 작은 쪽)를 택하는 것이 해석에도 유리하다. 추가로 각 군집별 실루엣 분포를 보아 "한 군집만 유난히 낮지는 않은지" 확인해 두면 안전하다.

5. Iris 데이터에 K-Means(k=3) 와 Agglomerative(linkage="ward", n_clusters=3) 를 각각 적용한 뒤, 두 결과의 라벨이 얼마나 일치하는지 sklearn.metrics.adjusted_rand_score 로 비교하라. 그리고 KMeans 의 라벨 번호와 Agglomerative 의 라벨 번호가 "같은 군집이라도 번호가 다를 수 있는" 이유를 한 줄로 설명하라.

5 번 해답

```
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.metrics import adjusted_rand_score

iris = load_iris()
X_s = StandardScaler().fit_transform(iris.data)

km = KMeans(n_clusters=3, n_init=20, random_state=42).fit(X_s)
agg = AgglomerativeClustering(n_clusters=3, linkage='ward').fit(X_s)

ari = adjusted_rand_score(km.labels_, agg.labels_)
print(f'KMeans vs Agglomerative ARI = {ari:.3f}')
```

Iris 의 경우 ARI 는 보통 0.6~0.8 수준으로 두 결과가 상당히 일치한다. KMeans 와 Agglomerative 가 같은 군집 분할을 했어도 라벨 번호가 다른 이유는, 두 알고리즘이 군집에 번호를 매기는 규칙이 다르기 때문이다 — KMeans 는 초기 중심값의 인덱스 순서대로, Agglomerative 는 덴드로그램의 병합 순서대로 라벨을 부여한다. ARI 는 이런 번호 순서와 무관하게 "같은 군집에 속한 쌍" 이 얼마나 일치하는지만 보기 때문에 라벨 비교에 적합하다.